

THE HONESTLY SERIES

Honestly,

CLAUDE CODE



An honest guide for humans



BRANDON R. HECKER



Real quick:

Yes, this is a guide for humans learning to use AI.

Yes, I used AI to write the guide.

Yes, that's hypocritical.

Mind your business.

CHAPTER 1 • ORIENTATION

01

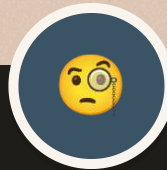
← you are here

So what the hell is Claude Code



You'll learn

- ✓ What it actually is
- ✓ What it isn't (and why)
- ✓ Why anyone bothers



Honest questions

- 1 Will it replace me?
- 2 Is my data safe?
- 3 What if it makes stuff up?
- 4 Am I going to burn out?

CONCEPT · SECTION 1 OF 9

What it actually is.

(short version: you type, claude does things.)

You're a little scared. Or skeptical. Or you've avoided this whole conversation for months because everyone around you sounds insufferable about AI. All reasonable.

This chapter is so we're talking about the same thing before you install anything.

Claude Code is a program you talk to in your computer's terminal and tell it to do things while looking like a hacker from TV. It can **read** your files, **write** new ones, **run** commands, and remember things between sessions.

It's not a website. It's not a copilot in your sidebar. It runs in a terminal window. The same place a developer would run code.

Underneath, it's **Claude** (the AI made by Anthropic) wired up to your computer in a useful way.

I PLAIN ENGLISH

A "terminal" is a text window. You type a sentence, the computer does something. Press **Enter** to send.



PLAIN ENGLISH

terminal: the text window. Also called "console" or "command line." Same thing.



PLAIN ENGLISH

Anthropic: the company that makes Claude.



ALSO HELPFUL

What it's *not*: **section 3**

Why bother: **section 4**



RABBIT HOLE

The word "shell" comes from the fact it wraps the kernel. You will never need to care about this.

TANGENT ↪ SKIPPABLE (it's fine to skip this, meet you on the next page)

About where this book actually lives.



The PDF in your hands is a **snapshot**. The real version of this book lives on the internet, in what engineers call a "repo." If you've heard "git repo" thrown around and felt left out, congrats, it's a folder. Engineers have a special word for folder. That's the whole mystery.

By the time you're reading this, the online version has probably gotten newer.



READ THE LATEST

github.com/brandonrhecker/honestly-claude-code

No account. No software. Pretend it's just a webpage. It is.



scan: repo

NO PANIC

*you don't need git
to read the book.*

What it's not.

four things, mostly

It helps to name what this isn't before naming what it is. Otherwise you'll keep importing baggage from the last AI thing you saw.



Not a chatbot.

A chatbot just talks. This does things. Edits files, runs commands, looks at git.



Not autonomous.

It asks permission before doing anything destructive. You're the human in the loop.



Not for engineers only.

You don't need to know how to code. You do need to be willing to type into a terminal. There's a difference.



Not magic.

It will get things wrong. Sometimes spectacularly. We'll cover how to catch it.

TAKEAWAY

*if it scares you,
good. that's
useful skepticism.*

CONCEPT · SECTION 4 OF 9

Why anyone bothers.

(it's not about the hype.)

You can hand off the tedious stuff. The "I have this file, can you rename it and update the references" stuff. The "organize my downloads folder by date" stuff. The "write me a checklist for the trip I'm planning" stuff.

It's not going to replace you. It's going to handle the work that's **beneath** you so you can do the work that **isn't**.

THE ONE-LINER*Boring work, gone. Real work, still yours.***FOR EXAMPLE**

Rename 200 photos by date. **ch. 03**

**FOR EXAMPLE**

Draft a checklist from a half-baked idea. **ch. 04**

**FOR EXAMPLE**

Organize a folder you've been avoiding. **ch. 03**

**BUT WAIT**

The next 4 sections are the honest ones. Read them.

— BUT WAIT · SECTION 5 OF 9

Am I going to lose my job?

Honest answer: maybe parts of it, probably not the whole thing.

The people who lean hardest into AI tools tend to climb faster, not get replaced. The people who refuse to touch it tend to get left behind. Neither's a great option.

What works best: learn enough to know what it's **good at** and what it **isn't**, then use it where it actually helps. Ignore the hype where it doesn't.

TL;DR

Refusing to engage is its own risk. So is going all-in. The middle path is boring and it works.

MORE IN CHAPTER 09

Will it replace me? The long version.



— BUT WAIT · SECTION 6 OF 9

Is my data safe?

Short version: your **conversations** go to Anthropic's servers to get processed. **Files** Claude reads stay on your computer unless you explicitly ask it to share them somewhere.

By default it **can't access the internet**. That's important. If you don't give it the wifi keys, it can't phone home with your stuff.

TL;DR

Conversations leave. Files stay. The thing has to ask you before it goes online.

MORE IN CHAPTER 09

Is my data safe? The long version.



— BUT WAIT · SECTION 7 OF 9

What if it just makes shit up?

It does, sometimes. **Confidently. With zero hedging.** That's the most important thing about it to remember.

The fix isn't to never use it. The fix is to never **trust it blindly**. Especially on facts, names, dates, code, or anything that matters.

You're the human. You're the check.

TL;DR

Yes, it lies. Confidently. Your job is to check. That part hasn't changed.

MORE IN CHAPTER 09

When it screws up. The long version.



— THE HONEST PART · SECTION 8 OF 9

About burnout.

Using AI is going to take mental energy you weren't spending before. Some days it'll save you hours. Some days it'll waste an hour and you'll feel dumb. That's normal.

If you start to feel burned out, **stop**. Use it less. Letting a tool run you is the same trap as the old job, just with a new tool.

TL;DR

It will save you time some days and cost you time others. If it starts owning you, walk away for a while.

MORE IN CHAPTER 09

Burning out in an automated world. The long version.



WHERE YOU ARE · SECTION 9 OF 9

Now you get it.

(if you're still here, you've got the gist.)

tl;dr

1 **Claude Code is an AI that runs in your terminal** and does real work on your files.

2 **It's not magic, it's not your replacement**, and it's not safe to trust blindly.

3 **It can save you real time on real tasks.**

4 **Your fears are legitimate; the answers are nuanced.**

Ready?

Chapter 2 installs the thing. One page at a time.



scan: ch.01

UP NEXT • CHAPTER 2

02

Get the damn thing running.

Install Claude Code on your machine. We do Mac, Windows, and Linux. Pick one, skip the rest.



STARTS NEXT



YOU WILL DO

Install Claude on your actual machine.

YOU WILL SEE

Claude say hello back in plain English.

DIFFICULTY

Still easy. Promise.

see you on the next page

honestly, **brandon**. still not an algorithm



repo • ch.02

CHAPTER 2 • SETUP

02

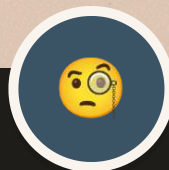
← you are here

Get the damn thing running



You'll do

- ☒ Install Node, then Claude Code
- ☒ Log in to Anthropic



Honest questions

- 1** Will my old laptop handle it?
- 2** What does it actually cost?
- 3** What can Claude see?

GROUND RULES · SECTION 1 OF 11

Will my crappy old laptop handle this.

probably, yes

You're putting Claude Code on your machine. After this chapter, you type `cLaude` into a terminal and the AI answers.

Probably, yes. Claude Code is a terminal app. The thinking happens on Anthropic's servers, not your computer. Your machine is just the keyboard and screen. If your laptop can run Chrome with a few tabs open without crying, it can run Claude Code.

REALISTIC BAR

✓ Made in the last ~8 years

✓ Mac, Windows 10/11, or Linux

✓ ~2 GB free disk

✓ A real internet connection (this is the bottleneck, not your CPU)

GROUND RULES · SECTION 2 OF 11

What it actually costs.

about \$20/month

Anthropic gives you a few options. The honest breakdown:

PLAN	COST	WHAT YOU GET
Free	\$0	Claude on the website only. No Claude Code.
Pro	~\$20/mo	Website + Claude Code, light use. Fine for this whole book.
Max (5×)	~\$100/mo	When you outgrow Pro and want more headroom.
Max (20×)	~\$200/mo	Power-user, daily-driver tier.
API	pay-per-token	Per-call billing. Not the simple path for beginners.

THE HONEST PICK*Pro, ~\$20/month. About the cost of a streaming subscription.***CHECK FIRST***prices shift.
claude.ai/pricing
is the truth.*

SETUP · SECTION 3 OF 11

Setting up your Anthropic account.

(already on Pro or higher? skip to the install.)

1

Open `claude.ai` in your browser and click **Sign up** (top right).

2

Sign up with Google for the fastest path, or use email and a password. Verify your email if it asks.

3

Once you're logged in, click your avatar (top right) → **Settings** → **Plans** (or **Billing**).

4

Pick **Pro**. Add a credit card.

5

Done. The **same account** works for both the `claude.ai` chat website and the Claude Code tool you're about to install.



PLAIN ENGLISH

account: one login. It covers the website and the terminal tool both.



HEADS UP

The **Free** plan won't run Claude Code. You need Pro or higher.



ALSO HELPFUL

What it costs: **section 1**



NO PANIC

*You can cancel any time.
Chapter 8 covers pulling the plug, billing included.*

What can Claude see on my machine?

Short version: only the folder you launch it from. When you type `c\laude` inside, say, your novel folder, that folder and its subfolders are Claude's whole world for that session. It can read those files, edit them, and run commands inside.

It **cannot** see your other folders, touch other apps, or rummage through your hard drive on its own.

TL;DR

It sees the folder you start it in. Nothing above it, nothing beside it.

MORE IN CHAPTER 09

Is my data safe? The long version.



INSTALL · SECTION 5 OF 11

Install on Mac.

five steps

1

Open **Terminal**: press `Cmd+Space`, type `Terminal`, hit Enter.

2

Install Node with NVM (a tool that installs Node for you). Paste this, then **close the Terminal completely and reopen it**.

bash

```
$ curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh | bash
```

3

Install the latest Node:

bash

```
$ nvm install --lts
```

4

Install Claude Code. The `-g` flag means "global", so it works from any folder.

bash

```
$ npm install -g @anthropic-ai/claude-code
```

5

Confirm it landed, then launch. First launch shows three prompts. **Turn to page 8**.

bash

```
$ claude --version
$ claude
expect something like 2.1.138 (Claude Code)
```


Install on Windows.

five steps

1

Open your browser, go to `nodejs.org`, and click the big green **Get Node.js** button. When the file downloads, open it and click **Next** through every screen without changing anything, then **Install**.

2

Open **Windows Terminal**: press the `Windows` key, type `Terminal`, hit Enter. A dark window opens. Keep it open.

3

Windows blocks scripts by default. Allow your own user to run them (one-time, standard dev setup). When it asks "Are you sure?", type `Y`.

Windows PowerShell

```
PS> Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

4

Install Claude Code. The `-g` flag means "global", so it works from any folder.

Windows PowerShell

```
PS> npm install -g @anthropic-ai/claude-code
```

5

Confirm, then launch. First launch shows three prompts. **Turn to page 8.**

Windows PowerShell

```
PS> claude --version
```

```
PS> claude
```

INSTALL · SECTION 7 OF 11

Install on Linux.

or WSL

On **WSL**? Follow this section, not the Windows one. And the browser login won't auto-open, so you'll copy the URL and code by hand.

1

Check if you already have Node. If it prints `v18` or higher, skip to step 3.

bash

```
$ node --version
```

2

No Node? Install via NVM. **Do not** `sudo apt install npm` on Ubuntu, it's outdated and breaks Claude. Close and reopen the terminal between the two commands.

bash

```
$ curl -o- https://raw.githubusercontent.com/nvm-  
sh/nvm/v0.40.4/install.sh | bash  
  
$ nvm install --lts
```

3

Install Claude Code:

bash

```
$ npm install -g @anthropic-ai/claude-code
```

4

Launch. First launch shows three prompts. **See the next page.**

bash

```
$ claude
```

The three prompts.

same on every OS

The very first time you run cLaude, it walks you through three quick things. Then you're at the prompt.

1

Pick a text theme. Arrow keys to highlight, Enter to select. Dark mode (the default) is fine. Change it later with `/theme`.

2

Log in to Anthropic. Claude prints a long URL. Copy it into your browser, log in, and the browser shows an authorization code. Copy that code, switch back to the terminal, paste it in, hit Enter.

3

Confirm the workspace. It shows the folder you launched in and asks if you trust it. Select **"Yes, I trust this folder."** This fires the first time you launch in any new folder, because Claude is about to read, edit, and run things there.

WELCOME

*you're officially
a nerd now. the
hard part's behind you.*

DEBRIEF · SECTION 9 OF 11

What just happened.

(you installed two things, that's it.)

Node.js, the runtime Claude Code is built on. You don't need to understand it, just know it's required.

Claude Code, the actual tool. When you first ran `c\laude` it logged you into Anthropic. From now on it remembers you.

You're in an interactive session now. The prompt is waiting. Don't type yet, that's the next chapter. For now, close it cleanly:

```
claude> /exit
```



PLAIN ENGLISH

Node.js: the engine Claude Code runs on. Install once, forget it.



PLAIN ENGLISH

npm: came with Node. It's how you installed Claude.



PLAIN ENGLISH

A **session** is one open conversation. **/exit** ends it.



UP NEXT

Make it do something real.
ch. 03

SECTION 10 OF 11

In case of emergency.

don't throw the laptop

Didn't work? You didn't do anything wrong. We'll get through this together.

IT SAYS	WHAT IT MEANS	WHAT TO DO
command not found	Node isn't on your PATH	Close and reopen the terminal. Still broken? The Node install didn't finish, re-run that step.
EACCES: denied	npm can't write where it's trying to	On Mac, use the NVM install above, not Homebrew or the .pkg . Don't use sudo .
scripts is disabled (Win)	You skipped the script-policy step	Run the Set-ExecutionPolicy command (Windows step 3), answer Y , then retry.
nvm: command not found	Your shell didn't reload	Close and reopen the terminal. New shells pick up the nvm config.
Login spins forever	Pop-up blocker or the tab failed to open	Copy the URL the terminal printed and open it manually in a fresh browser tab.
Anything else	The internet is a big place	Paste the exact error into chat at claude.ai : "what does this mean on [your OS]?" Yes, use Claude to debug Claude.

WHERE YOU ARE · SECTION 11 OF 11

Now you can.

(the rest of the book is built on top of this.)

tl;dr

1 Open a terminal and type `claude` to start a session.

2 See the interactive prompt.

3 Exit cleanly with `/exit`.

That's the whole install.

The rest of the book is built on top of this.



scan: ch.02

UP NEXT • CHAPTER 3

03

*Make it do
something for you.*

Your first real task. You point Claude at a folder and it writes you an actual file. No coding.



STARTS NEXT



YOU WILL DO

Hand Claude a plain-English task.

YOU WILL SEE

A real file appear on your machine.

DIFFICULTY

Easier than the install.
Promise.

*see you on the next page*honestly, **brandon**. still not an algorithm

repo • ch.03

CHAPTER 3 • YOUR FIRST TASK

03

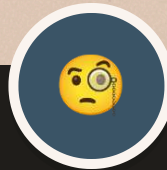
← a packing list, specifically

Make it do something.



You'll do

- ✓ Launch Claude in a folder
- ✓ Give it a real task
- ✓ Read, approve, and find the file



Honest questions

- 1 Is a packing list all it can do?
- 2 What if it writes something wrong?
- 3 Does the file stay after I exit?

CONCEPT · SECTION 1 OF 8

Why bother making Claude do your dirty work.

(the "oh, it's real" moment.)

You installed Claude Code. Now you'll have it do an actual thing. Not a hello-world, not a placeholder. Something you might actually want.

This chapter has Claude write you a packing list. Why packing? Because everyone's made one, it needs zero setup, and you'll end up with a real file you can use.

Until you watch Claude do something useful on your actual computer, the whole thing is theoretical. This is the "oh, it's real" moment.

It's also the smallest possible demo of the loop: you ask, Claude proposes, you approve, Claude writes. Every more-complex thing in the rest of the book is just this with more sophistication.

THE LOOP

*You ask. Claude proposes. You approve.
Claude writes.*



IS THIS ALL

No. A packing list is the smallest useful demo. You'll go bigger in chapter 04 and beyond.



WHAT IF IT'S WRONG

Read it before you approve. If it's wrong, say so. Claude will fix it. You're the check.



ALSO HELPFUL

What .md files are: [section 2](#)

What just happened: [section 6](#)

CONCEPT · SECTION 2 OF 8

First, a word about .md files.

(plain text. less scary than they look.)

Claude is about to write a .md file. If these have ever intimidated you, that's fine. They're just plain text with lightweight formatting: # for headers, - for bullets, ****bold**** for bold.

Your default text editor opens it just fine. You'll see the # and - characters mixed in. Still readable. For advanced readers: VS Code shows a rendered preview side by side.

OPENED IN	WHAT YOU SEE
Notepad / TextEdit	Raw: # and - mixed with text. Still readable.
VS Code + preview	Pretty rendered side by side. Free, easiest path.
GitHub (if you upload)	Pretty rendered automatically.
Obsidian / Bear / Notion	Pretty rendered. Pick whichever you use.



PLAIN ENGLISH

markdown: a simple system for adding formatting to plain text. Invented so humans can write formatted docs without HTML.



RABBIT HOLE

This entire book is written in Markdown. If you're reading it on GitHub, you're seeing it render right now.



DO YOU NEED VS CODE

No. Notepad and TextEdit work fine. VS Code just makes it nicer to read.

SETUP · SECTION 3 OF 8

Make the folder.

brand new, just for this

STEP 1 · OPEN YOUR TERMINAL

MAC

Cmd+Space, type Terminal,
hit Enter.

WINDOWS

Windows key, type
Terminal, hit Enter.

LINUX

You already know.

STEP 2 · MAKE YOUR FOLDER

MAC / LINUX / WSL

bash

```
$ mkdir -p ~/claude-  
experiments/packing-list
```

WINDOWS

Windows PowerShell

```
PS> mkdir ~\claude-  
experiments\packing-list -  
Force
```

→ Then enter the folder in the same terminal window:

STEP 3 · ENTER IT

MAC / LINUX / WSL

bash

```
$ cd ~/claude-  
experiments/packing-list
```

WINDOWS

Windows PowerShell

```
PS> cd ~\claude-  
experiments\packing-list
```

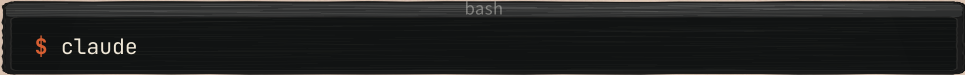
→ `~` is your home folder. `-Force` means don't complain if it already exists.

SETUP · SECTION 4 OF 8

Launch Claude. Give it a job.

the actual moment

STEP 4 · LAUNCH CLAUDE

A terminal window with a dark background. The prompt 'bash' is visible at the top right. The command '\$ claude' is entered at the prompt.

First time in this folder, Claude asks if you trust it. Pick **Yes, I trust this folder** and press Enter. One-time check per folder.

STEP 5 · GIVE IT A TASK

PASTE THIS INTO CLAUDE

Write me a packing list for a 4-day trip to a warm climate. Essentials only: clothes, toiletries, electronics, paperwork. Save it as packing-list.md in this folder.

Change the trip details to something you're actually planning, or keep it as-is. Either way works.

SETUP · SECTION 5 OF 8

Say yes. Find the file.

it actually exists now

STEP 6 · READ AND APPROVE

Claude shows you the packing list and asks permission to save it. **Read it.** If something looks off, tell Claude what to fix before approving. When it looks good, approve.

STEP 7 · FIND IT IN YOUR FILE MANAGER

MAC

Open Finder. Press `Cmd+Shift+G`, paste:

```
~/claude-experiments/packing-  
list
```

WINDOWS

Press `Win+R`, paste:

```
%USERPROFILE%\claude-  
experimentspacking-list
```

Double-click `packing-list.md` to open it. On Windows, click **Open anyway** if it warns about file type. For a prettier view, open it in VS Code.



The file exists. On your disk. You didn't write a line of code.

CONCEPT · SECTION 6 OF 8

What just happened.

(you just did the whole game.)

STEP 8 · EXIT CLAUDE

```
bash
$ /exit
```

Five things happened in that sequence:

- 1 Launched Claude in a specific folder** Claude can only see and write to the folder you launched from (and subfolders).
- 2 Gave it a real prompt** not a test. An actual thing.
- 3 It proposed an action (writing a file)** and waited for your okay.
- 4 You approved** and it did the thing.
- 5 The file exists** On your disk. Yours to edit, delete, or send.

The "ask permission before writing" step is core to how Claude Code works. It's not autonomous. You're always the human in the loop. That's a feature, not a bug.



PLAIN ENGLISH

approve: Claude always shows you what it plans to do before doing it. Your OK is required. This is on purpose.



NEXT STEP

Next chapter: give Claude a memory. It's a bigger shift than it sounds.

IN CASE OF EMERGENCY · SECTION 7 OF 8

Did Claude get weird on you?

not your fault. let's fix it

It happens. Here's what's probably going on and how to get back on track.

MESSAGE OR SYMPTOM	WHAT IT MEANS	WHAT TO DO
<code>claude: command not found</code>	The install didn't take.	Re-run chapter 02 from your OS section.
Claude wrote the file in the wrong place	You launched from the wrong folder.	Run <code>/exit</code> , then <code>cd</code> to the right folder and launch again.
Claude refused to write	A permission setting may be blocking it.	Run <code>/permissions</code> inside Claude to see what's allowed.
Claude wrote something weird	The model is good but not magic.	Tell it exactly what to fix. Be specific. It will re-do it.
You can't find the file	Probably in a different folder than you think.	Run <code>pwd</code> in the terminal. That's the folder Claude was working in.



WHERE YOU ARE · SECTION 8 OF 8

Now you can.

tl;dr

1 Launch Claude in a folder and have it write files there

2 Read a Claude action proposal and decide whether to approve

3 Find and use the files Claude creates on your disk

4 Trust that Claude isn't going to do something destructive without asking first

Three chapters down. The next one teaches Claude where it is and what your project is about. A much bigger shift than it sounds.

UP NEXT • CHAPTER 4

04

Give it a brain.

Write a CLAUDE.md that tells Claude what a folder is for.
Claude reads it automatically every session.



STARTS NEXT



YOU WILL DO

Write a CLAUDE.md and watch
Claude follow it.

YOU WILL SEE

Claude producing work that
actually matches your rules.

DIFFICULTY

Same as this one. Less
terminal, more thinking.

*see you on the next page*honestly, **brandon**. still not an algorithm

repo • ch.04

CHAPTER 4 • MEMORY

04

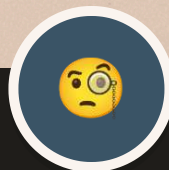
← just a text file, actually

Give it a
brain.



You'll do

- ✓ Write your first CLAUDE.md
- ✓ Give a folder a brain
- ✓ Watch Claude actually follow your rules



Honest questions

- 1 Will Claude actually read it?
- 2 What should I put in it?
- 3 Can I break something with it?

CONCEPT · SECTION 1 OF 9

Why bother with **CLAUDE.md**.

(because repeating yourself forever is not a feature.)

In chapter 03, you launched Claude in a folder and asked for a packing list. It worked, but Claude went in blind. It didn't know what the folder was for, what you preferred, or what rules to follow.

That's where CLAUDE.md comes in. It's the file you write that tells Claude what the folder is for, who you are, and the rules to follow inside it.

This chapter: you will create your first CLAUDE.md and watch Claude actually follow it.

**WITHOUT IT**

Claude forgets everything between sessions. You re-explain the project every time.

**WITH IT**

Claude reads the file when you launch. Acts like someone who's been on the project for weeks.

**ALSO HELPFUL**

What to put in it: **section 2**

If it breaks: **section 8**

CONCEPT · SECTION 2 OF 9

First, a word about **CLAUDE.md**.

(a text file with a specific name. that's it.)

It's a `.md` file named `CLAUDE.md` (uppercase, exactly). Lives in the root of whatever folder you launch Claude from. One per project.

Claude reads it automatically every time you launch. You don't tell it to. It just does.

Think of it as the folder's BELIEVE sign. You write it once and every session loads it automatically.

I PLAIN ENGLISH

"Root of the folder" means the main folder itself, not a subfolder inside it. If your folder is `writing/`, `CLAUDE.md` sits directly inside `writing/`.



WHAT TO PUT IN IT

What the project is for. Your preferences. Style rules. Filename formats. Things to never do.



HOW LONG

Aim for 200-400 words. Too long and Claude has too much to weigh. Bullet points beat paragraphs.



RABBIT HOLE

You're using `CLAUDE.md` right now. This entire book has one that tells Claude the voice, tone, and banned-word list.

SETUP · SECTION 3 OF 9

Make your folder.

one command

Open your terminal. Pick your OS.

STEP 1 · MAKE YOUR FOLDER

MAC / LINUX / WSL

bash

```
$ mkdir -p ~/claude-  
experiments/writing
```

WINDOWS

Windows PowerShell

```
PS> mkdir ~\claude-  
experiments\writing -Force
```

STEP 2 · ENTER IT

MAC / LINUX / WSL

bash

```
$ cd ~/claude-  
experiments/writing
```

WINDOWS

Windows PowerShell

```
PS> cd ~\claude-  
experiments\writing
```

STEP 3 · LAUNCH CLAUDE

bash

```
$ claude
```

REMINDER*same terminal
from chapter 02.
it's still open.*

First time in this folder, Claude asks if you trust it. Pick **Yes, I trust this folder** and press Enter.

SETUP · SECTION 4 OF 9

Ask Claude to write its own brain.

yes, really

You're going to ask the AI to write its own instructions. This is not weird. This is the move.

STEP 4 · PASTE THIS

PASTE THIS INTO CLAUDE

Write a CLAUDE.md for this folder. This is where I'll keep my personal writing: short essays and journal entries. My writing voice is Ted Lasso: relentlessly warm, folksy wisdom, finds the good in everyone, with occasional mustache-based confidence. Around 400-600 words per piece. Save each piece as YYYY-MM-DD-slug.md.

STEP 5 · APPROVE IT

Claude will write a CLAUDE.md and ask permission to save it. Read through what it wrote. If it looks reasonable, approve. You can edit it later, it's just a text file.

SETUP · SECTION 5 OF 9

Put it to work.

now Claude knows the rules

Claude just read its own CLAUDE.md. Let's see if it sticks.

STEP 6 · PASTE THIS

PASTE THIS INTO CLAUDE

Write a pep talk for someone who's scared to start using AI. Follow the rules in CLAUDE.md.

STEP 7 · APPROVE IT

Claude reads CLAUDE.md, follows the style, keeps the length, and saves the file with the `YYYY-MM-DD-slug.md` format. Approve when ready.

If something looks off (wrong length, wrong vibe), open CLAUDE.md in a text editor and tighten the rules. More specific beats more verbose.

WHAT TO WATCH

*style matches
Ted Lasso?
filename format right?*

SETUP · SECTION 6 OF 9

Find your file.

it actually exists now

Step 8: open the folder in your file browser and find the new file.

MAC

Open Finder. Press **Cmd+Shift+G**, paste:

```
~/claude-experiments/writing
```

Double-click the `YYYY-MM-DD-slug.md` file to open it in TextEdit.

WINDOWS

Press **Win+R**, paste:

```
%USERPROFILE%\claude-experiments\writing
```

Press Enter. File Explorer opens to the folder. Double-click the file to open it in Notepad.

LINUX / WSL

Run `ls ~/claude-experiments/writing` to confirm the file is there. Open it with `cat FILENAME.md` or your text editor of choice.



The file exists. Claude wrote it. You didn't write a line of code to make that happen.

CONCEPT · SECTION 7 OF 9

What just happened.

(you made a folder with a memory.)

STEP 9 · EXIT CLAUDE

```
bash
$ /exit
```

Here's what actually happened in those 9 steps:

- 1 **You created a project-level CLAUDE.md** that defines what this folder is for and how Claude behaves inside it.
- 2 **Claude read it automatically** when you sent the second prompt. You didn't tell it to.
- 3 **It followed the rules** the style, the word count, the filename format.
- 4 **This works forever** every future session in this folder loads CLAUDE.md automatically.



PLAIN ENGLISH

session: one run of Claude Code, from launch to exit. Each session starts fresh unless CLAUDE.md tells it otherwise.



NEXT STEP

You can edit CLAUDE.md directly any time. Add rules, remove rules, refine the style. Claude picks up the changes next launch.

IN CASE OF EMERGENCY · SECTION 8 OF 9

Be a goldfish.

don't dwell. fix and move on

CLAUDE.md not behaving? Here's what's probably happening and how to fix it.

SYMPTOM	WHAT IT MEANS	WHAT TO DO
Claude wrote something that ignores your CLAUDE.md	Either the file is misnamed (must be <code>CLAUDE.md</code> exactly, uppercase) or you launched Claude in a different folder	Run <code>ls</code> to see the file list. Check that <code>CLAUDE.md</code> is there. Confirm you're in the right folder with <code>pwd</code> .
You can't find CLAUDE.md after creating it	Probably saved in the wrong folder	Type <code>pwd</code> to see where Claude was working. That's where the file is.
Claude follows some rules but not others	Rules might be ambiguous, contradictory, or buried in long paragraphs	Open <code>CLAUDE.md</code> and rewrite the conflicting parts as a bulleted list. Shorter, more specific.
CLAUDE.md is too long and Claude seems to ignore parts	Lots of context means Claude has more to weigh	Trim to the essentials. Aim for 200-400 words.

Now you can.

tl;dr

1 Create a `CLAUDE.md` in any folder to define what it's for

2 Have Claude read project context automatically every session

3 Edit `CLAUDE.md` to refine rules over time

4 Know that "Claude doesn't remember" is half-true: it doesn't remember conversations, but a project's `CLAUDE.md` is forever

You can do this. Onward. Four chapters down. Next: skills. We'll install someone else's workflow and use it in this same folder.

UP NEXT • CHAPTER 5

05

Bolt on a skill.

Install someone else's workflow and use it in this same folder. No coding, no config files.



STARTS NEXT



YOU WILL DO

Install a skill and run it on a real task.

YOU WILL SEE

Someone else's workflow running in your folder.

DIFFICULTY

About the same as this one.

see you on the next page

honestly, **brandon**. still not an algorithm



repo • ch.05

CHAPTER 5 • POWER-UPS

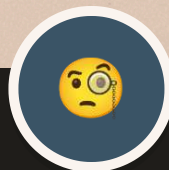
05

← superpowers, specifically

Bolt on a skill.

You'll do

- ✓ Register a plugin marketplace
- ✓ Install the superpowers plugin
- ✓ Use a brainstorming skill on a real problem



Honest questions

- 1 Is this safe to install?
- 2 Do I have to keep it?
- 3 What else can I get?

CONCEPT · SECTION 1 OF 9

Why bother bolting on skills.

(someone already built this.)

You've installed Claude Code, made it write a file, and taught it about your project.

This chapter: we'll install superpowers (the #1 community Claude Code plugin, by Jesse Vincent), use its brainstorming skill on a weekend meal-prep routine you'll actually stick with, then we'll uninstall the whole thing. That last part is also a skill worth having.

You just taught Claude about your project with `CLAUDE.md`. That's project context. Skills are a different layer: pre-built and reusable workflows that work in ANY project.

CLAUDE.md = what your project is

Skills = techniques anyone can use

Both = can be active at once

A skill is a pre-built mini-instruction set you install once and trigger with a slash command (`/skill-name`). Like keyboard shortcuts for whole multi-step processes.

The win: when someone smart has already designed how to do something well, you install their work and use it across all your projects.



IS THIS SAFE

Plugins run inside Claude Code. They can't access your machine, install software, or phone home. They're just instruction sets.



ALSO HELPFUL

How to uninstall: [section 8](#)

What else is out there: [section 7](#)

CONCEPT · SECTION 2 OF 9

First, a word about skills and marketplaces.

(two concepts. then you're done.)

MARKETPLACE

A place that lists plugins. Usually a GitHub repo. You register a marketplace with Claude Code once, then you can install any plugin it contains.

PLUGIN

A bundle of skills. Some plugins contain a single skill. Many contain a dozen or more bundled together. When you install a plugin, you get all its skills at once.

THE INSTALL FLOW · EVERY TIME

- 1 **Register the marketplace** — (one time).
- 2 **Install the plugin from it.** —
- 3 **Reload so Claude Code picks up the new skills.** —



WHY RELOAD

Claude Code doesn't scan for new plugins automatically. The reload command tells it to pick up whatever changed.



ONE MARKETPLACE

You only need to register a marketplace once. After that, you can install any plugin from it anytime.

SETUP · SECTION 3 OF 9

Make the folder.

same pattern as before

STEP 1 · OPEN YOUR TERMINAL

MAC

Cmd+Space, type Terminal,
hit Enter.

WINDOWS

Windows key, type
Terminal, hit Enter.

LINUX

You already know.

STEP 2 · MAKE YOUR FOLDER

MAC / LINUX / WSL

bash

```
$ mkdir -p ~/claude-  
experiments/brainstorm
```

WINDOWS

Windows PowerShell

```
PS> mkdir ~\claude-  
experiments\brainstorm -  
Force
```

Then enter the folder in the same terminal window:

STEP 3 · ENTER IT

MAC / LINUX / WSL

bash

```
$ cd ~/claude-  
experiments/brainstorm
```

WINDOWS

Windows PowerShell

```
PS> cd ~\claude-  
experiments\brainstorm
```

STEP 4 · LAUNCH CLAUDE CODE

bash

```
$ claude
```

SETUP · SECTION 4 OF 9

Register it. Install the skill.

STEP 4 · REGISTER THE MARKETPLACE

bash

```
$ /plugin marketplace add obra/superpowers
```

Tells Claude Code where to look for plugins. Nothing installs yet.

STEP 5 · INSTALL SUPERPOWERS

bash

```
$ /plugin install superpowers@superpowers-dev
```

The name became `superpowers-dev` — that's what the marketplace declares itself as. Confusing, normal.

STEP 6 · PICK WHERE TO INSTALL IT

Claude Code will ask. Pick the first option:

- ☒ **Install for you (user scope)** — available in every folder you launch Claude from ← pick this
- ☐ Install for this project only
- ☐ Install for teammates (shared via git)

SETUP · SECTION 5 OF 9

Reload. Then brainstorm.

the actual moment

STEP 7 · ACTIVATE THE NEW SKILLS

```
bash  
$ /reload-plugins
```

Tells Claude Code to pick up the plugin you just installed. Do this after every install.

STEP 8 · INVOKE THE BRAINSTORMING SKILL

PASTE THIS INTO CLAUDE

```
/brainstorming
```

```
Help me brainstorm a weekend meal-prep routine I'll actually stick with.  
Ask me 3 clarifying questions max. Then propose one approach in a few  
bullets. Skip writing a separate design doc.
```

The "3 questions max" guardrail keeps this from turning into a 20-turn conversation. Answer the questions naturally — your schedule, what you usually eat, what's gone wrong before.

CONCEPT · SECTION 6 OF 9

What just happened.

(you installed a superpower.)

STEP 9 · EXIT CLAUDE

```
bash  
$ /exit
```

You just installed someone else's workflow, used it for something in your actual life, and you can uninstall it whenever you want. That's the move.

Four things happened in sequence:

- 1 **You registered a marketplace** — Claude Code now knows where Jesse Vincent's plugins live.
- 2 **You installed superpowers** — ~20 skills became available: brainstorming, debugging, TDD, and more.
- 3 **You reloaded** — activated the new skills without restarting your session.
- 4 **You invoked a skill** — with a scoped prompt to keep it short. The skill walked you through a structured brainstorm.



PLAIN ENGLISH

invoke: to call a skill by typing its slash command. Same idea as running a function — you trigger a pre-written workflow.



THE PATTERN

Register marketplace → install plugin → reload → invoke. Same flow every time, any plugin.

Where to find more skills.

the ecosystem

Now that you know the flow, you can browse what else is out there.

WHERE	WHAT'S THERE
<u>anthropics/claude-plugins-official</u>	Anthropic's official marketplace. Usually pre-registered. Browse the plugins/ folder to see the list.
<u>obra/superpowers</u>	The plugin you just installed. Browse the skills inside to see what else came along for the ride.
Search GitHub	Search for repos containing .claude-plugin/marketplace.json . The ecosystem is decentralized.

When you find one worth trying, it's the same flow every time: `/plugin marketplace add` → `/plugin install` → `/reload-plugins`.

HEADS UP

*third-party plugins
aren't vetted by
Anthropic. use judgment.*

GROUND RULES · SECTION 8 OF 9

How to uninstall.

when you're done

OPTION A · REMOVE THE PLUGIN

STEP 1

```
bash
$ /plugin uninstall
superpowers@superpowers-dev
```

Then reload:

STEP 2

```
bash
$ /reload-plugins
```

Skills gone. Marketplace stays registered.

OPTION B · REMOVE EVERYTHING

STEP 1

```
bash
$ /plugin marketplace remove
superpowers-dev
```

Then reload:

STEP 2

```
bash
$ /reload-plugins
```

Marketplace and all plugins gone. Total reset.

This is a normal part of trying new things. Don't keep stuff installed if you don't actually use it.

Install didn't go smoothly? You're not the first. Probably one of these.

SYMPTOM	WHAT IT MEANS	FIX
<code>/plugin marketplace add</code> says "not found"	Repo name is misspelled or doesn't exist.	Re-paste exactly: <code>/plugin marketplace add obra/superpowers</code>
Install said success but <code>/brainstorming</code> doesn't work	You skipped the reload.	Run <code>/reload-plugins</code>
Brainstorming won't stop asking questions	The skill pulls toward thoroughness.	Answer them, or say "let's wrap up." It'll comply.



WHERE YOU ARE · SECTION 9 OF 9

Now you can.

tl;dr

1 Add any marketplace to Claude Code.

2 Install plugins from a marketplace.

3 Invoke skills with `/skill-name`.

4 Uninstall plugins or whole marketplaces cleanly.

5 Browse the ecosystem to find more skills.

Five chapters down. Next: MCP, the thing that lets Claude talk to other apps you already use.

UP NEXT • CHAPTER 6

06

*Plug it into
your other apps.*

Connect Claude to tools you already use. One command installs a real MCP server.



STARTS NEXT



YOU WILL DO

Add an MCP server and use it to pull data from an app you already have.

YOU WILL SEE

Claude reading from a real external source mid-conversation.

DIFFICULTY

One command. Slightly more moving parts than this chapter.

*see you on the next page*honestly, **brandon**. still not an algorithm

repo • ch.06

CHAPTER 6 • CONNECTIONS

06

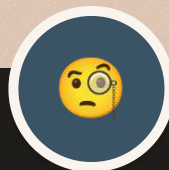
← your stuff. finally.

*Plug it into
your other apps.*



You'll do

- ✓ Register the Pocket Casts MCP
- ✓ Verify it works with /mcp
- ✓ Ask Claude to find podcast recommendations



Honest questions

- 1 Is this harder than plugins?
- 2 Do I need a Pocket Casts account?
- 3 What else can I connect?

CONCEPT · SECTION 1 OF 9

Why bother plugging it into other apps.

(one chapter from the finish line.)

You've installed Claude Code, made it write a file, given it project context with `CLAUDE.md`, and bolted on a skill. Claude already has built-in web tools: it can fetch a URL or search the web on its own.

What it CAN'T do natively: talk to specific apps and services. Your podcast app, your calendar, your notes app, your inbox. Claude doesn't know how to reach any of those unless you build it a bridge.

MCP is that bridge.

This chapter: we'll install the Pocket Casts MCP so Claude can recommend podcasts based on what you're learning. We'll point it at this book's repo URL and ask for episodes that'd help you keep going. Then we'll uninstall.



PLAIN ENGLISH

MCP: Model Context Protocol. The standard for "let an AI assistant talk to outside apps." Anthropic created it. Lots of tools support it now.



ALSO HELPFUL

What MCP actually is:

section 3

Where to find more:

section 8

CONCEPT · SECTION 2 OF 9

So why bother with MCP.

(generic vs. specific.)

Claude can already fetch URLs and search the web with its built-in tools. So why bother with MCP?

Built-in tools are generic. They know "the web" but not "your podcast app," "your calendar," "your inbox," "your CRM." Those are specific services with their own data and their own way of doing things.

MCP gives Claude specific connections to specific services. An MCP server is a small program that knows how to translate between Claude and one particular service. Install the Pocket Casts MCP and Claude can search Pocket Casts' catalog. Install the Slack MCP and Claude can read your Slack messages. And so on.

The win: stop being limited to what Claude was born knowing. Plug it into your stuff.

[↗](#) ALSO HELPFUL

What MCP actually is:

[section 3](#)

Where to find more:

[section 8](#)

CONCEPT · SECTION 3 OF 9

First, a word about MCP.

(three things to know.)

MCP = Model Context Protocol. It's the standard for "let an AI assistant talk to outside apps." Anthropic created it. Lots of tools support it now.

THE WAY IT WORKS

- 1 **Someone writes a small program** — (an MCP server) that knows how to talk to one specific thing: an app, a database, a website, whatever.
- 2 **You register that server** — with `claude mcp add`. Claude Code now has a bridge to that thing.
- 3 **Claude can reach it** — from any folder, through natural conversation — not a slash command.

A FEW HONEST NOTES

MCPs are more technical to set up than skills. This is the most technical chapter in the book. Take it slow.

Many MCPs need an API key or login. Some are open and free. We're using a free, no-login one today (Pocket Casts).

An MCP only adds the connection. Whether Claude uses it well depends on how you ask.



PLAIN ENGLISH

MCP server: a small program that translates between Claude and one specific service. You install the bridge; Claude uses it naturally.



VS. SKILLS

Skills extend what Claude *knows how to do*. MCPs extend what Claude can *reach*. Both are useful. Different jobs.

SETUP · SECTION 4 OF 9

Register the Pocket Casts MCP. *one command*

Open your terminal (same one from chapter 02). Paste this in and press Enter:

STEP 1 · REGISTER THE MCP

```
bash
$ claude mcp add --transport http pocketcasts
https://mcp.pocketcasts.com
```

Reads as: "Register an MCP called `pocketcasts` that lives at `https://mcp.pocketcasts.com`. Use HTTP transport."

WHAT'S HAPPENING HERE

- **No API key** — no signup, no auth. Pocket Casts hosts the MCP for free.
- **Nothing installs locally** — the MCP just points at their hosted server.
- **Works from any folder** — registered once, available everywhere you launch Claude.

SAME COMMAND

*Mac, Linux,
WSL, Windows —
all the same.*

SETUP · SECTION 5 OF 9

Launch it. Verify it.

confirm the bridge is up

STEP 2 · LAUNCH CLAUDE CODE

```
bash
$ claude
```

First time in this folder, Claude asks if you trust it. Pick **Yes, I trust this folder** and press Enter.

STEP 3 · VERIFY THE CONNECTION

```
bash
$ /mcp
```

You should see **pocketcasts** in the list. Press `Esc` to exit the MCP view and get back to the chat prompt.

If you don't see `pocketcasts`, jump to "in case of emergency" on page 10.

SETUP · SECTION 6 OF 9

Ask Claude.

the actual moment

STEP 4 · FETCH THE REPO AND ASK

PASTE THIS INTO CLAUDE

Fetch <https://github.com/brandonrhecker/honestly-claude-code> and use the Pocket Casts MCP to find 5 podcast episodes that would help me keep learning about the topics in this book.

Yes, the URL is this book's own repo. Meta, but it makes the demo concrete since you already know what's in it. Watch Claude work:

- It uses built-in WebFetch to read the repo.
- It pulls out the main topics (Claude Code, AI for non-engineers, learning AI tools, etc.).
- It uses the Pocket Casts MCP to search podcasts on those topics.
- It returns a list of episodes with titles and where to listen.

STEP 5 · EXIT CLAUDE

```
bash
$ /exit
```

You just had Claude combine its built-in web tools with a brand-new MCP connection to deliver something neither could do alone. That's the actual point of MCP.

CONCEPT · SECTION 7 OF 9

What just happened.

(two tools. one prompt.)

You did three things:

- 1 Registered an MCP server** — with `claude mcp add` .
Claude Code now knows there's a bridge to Pocket Casts' podcast catalog.
- 2 Verified it** — with `/mcp` inside Claude Code.
- 3 Used two tools together** — in one prompt: built-in WebFetch (to read the repo) AND the Pocket Casts MCP (to find related episodes).

The pattern for any MCP is the same: register it with `claude mcp add` , verify with `/mcp` , then use it through natural conversation. You don't invoke MCPs with slash commands the way you invoke skills. Claude reaches for them when relevant.

The Pocket Casts MCP we picked is unusually frictionless: free, no auth, hosted by Pocket Casts themselves. Most MCPs need an API key or an account login. That's just the trade. A service that knows about your data needs your permission to read it.



PLAIN ENGLISH

invoke: MCPs aren't invoked by you. You describe what you want. Claude decides whether to reach for the MCP. Different from skills, which you trigger with a slash command.



THE PATTERN

Register → verify with `/mcp` → use in conversation.
Same flow every time, any MCP.

Where to find more MCPs.

the ecosystem

Now that you know the flow, you can browse what else is out there.

WHERE	WHAT'S THERE
modelcontextprotocol/servers	Official MCP servers from the protocol team. Fetch, time, weather, filesystem, git, memory, and more.
Search GitHub	Search for <code>awesome-mcp-servers</code> . Community-maintained lists across many categories.
modelcontextprotocol.io	The official MCP docs.

When you install a new MCP, the pattern is the same as today: register the server with `claude mcp add`, confirm with `/mcp`, then use it.

HEADS UP

*third-party MCPs
aren't vetted by
Anthropic. use judgment.*

GROUND RULES · SECTION 9 OF 9

How to remove it.

and if it gets weird

WHEN YOU'RE DONE EXPERIMENTING

bash

```
$ claude mcp remove pocketcasts
```

Claude Code forgets the MCP. Nothing else changes on your machine. Pocket Casts itself wasn't installed locally; the MCP just pointed at their hosted server. No `/reload` step, no cleanup. Cleaner than plugins.

Welcome to your first MCP install. Things can go sideways. Probably one of these.

SYMPTOM	WHAT IT MEANS	FIX
<code>claude mcp add</code> says it can't find <code>claude</code>	Claude Code isn't on your PATH.	Close and reopen the terminal. Run <code>claude --version</code> to confirm.
<code>/mcp</code> shows pocketcasts as "failed" or "disconnected"	The HTTP endpoint isn't responding.	Try again in a minute. If still down, <code>claude mcp remove pocketcasts</code> outside Claude and re-add.
Claude says it can't find any podcasts	Maybe the topic search came up empty.	Try a more general topic, or ask Claude to "tell me what topics you extracted from the URL" so you can refine your ask.
Claude lists shows but no specific episodes	Pocket Casts searches by topic then lists recent episodes from matching shows. Sometimes shows don't have recent matches.	Ask Claude to "list 3 recent episodes from [show name]" to drill down.
You changed your mind	Easy.	<code>claude mcp remove pocketcasts</code> and you're back to neutral.

WHERE YOU ARE · SECTION 9 OF 9

Now you can.

1 Register an MCP server with `cLaude`
`mcp add`

2 Confirm it's working with `/mcp`

3 Use built-in tools and MCPs together in one prompt

4 Remove MCPs cleanly with `cLaude`
`mcp remove`

5 Know where to find more MCPs when you need them

Six chapters down. One short closing chapter to go.

UP NEXT • CHAPTER 7

07

*You made it.
Now what.*

Where to go from here. What to build next. How to keep getting better at this.



STARTS NEXT



YOU WILL SEE

A clear picture of where your new skills can take you from here.

YOU WILL GET

Honest advice on what to try next and how to keep learning without burning out.

DIFFICULTY

No new commands. Just perspective.

see you on the next page

honestly, **brandon**. still not an algorithm



repo • ch.07

CHAPTER 7 • CLOSING TIME

07

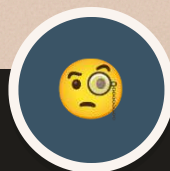
← you actually did it.

*You made it.
Now what.*



You'll learn

- ✓ Install Claude Code on any machine
- ✓ Make Claude do real work for you
- ✓ Teach it your project with CLAUDE.md
- ✓ Add skills and plug in apps via MCP



Honest questions

- 1 Where to take it from here
- 2 Honest translations for when "AI" comes up
- 3 The one thing the book can't teach you

CONCEPT · SECTION 1 OF 4

What you can do now.

You did it. That's all, folks. You made it through to the end. While people on LinkedIn are still "leveraging AI for synergistic outcomes," you actually know how the thing works.

Here's what you can do that they can't:

- ✓ **Install Claude Code.** on any computer that lets you install things.
- ✓ **Make Claude do real work.** write files, organize folders, draft the thing you've been procrastinating on.
- ✓ **Teach Claude about a project.** with CLAUDE.md, so you stop re-explaining yourself every session.
- ✓ **Install other people's skills.** with a couple of commands. Borrow what smarter people figured out.
- ✓ **Plug Claude into outside apps.** with MCP, when the built-in tools aren't enough.

While people on LinkedIn are still "leveraging AI for synergistic outcomes," you actually know how the thing works.



WHAT THIS MEANS

You can hold your own when "AI" comes up. Not because you memorized a definition. Because you've actually used it.



NEXT UP

Honest translations for when someone says "AI":

section 2

CONCEPT · SECTION 2 OF 4

When someone says **"AI."**

(honest translations.)

You can now hold your own. Some translations worth keeping:

"AI-POWERED"

Usually means Claude, GPT, Gemini, Grok (or whatever the newest model is when you're reading this) wrapped around something. The "AI" part is the model. The rest is a product someone built on top of it.

"WILL AI TAKE MY JOB?"

Depends on whether you understand it. You're now in the camp that does. The people who get displaced first are the ones treating it like magic they don't need to understand.

WHAT SETS YOU APART NOW

You know the difference between a prompt, a skill, an MCP, and a CLAUDE.md. You don't have to be the loudest person in the room about it. You just have to be the one who knows.



PLAIN ENGLISH

AI-powered: a product with a language model doing some of the work. The model is rented from Anthropic, OpenAI, Google, or similar. The product is what you're actually using.



NEXT UP

Where to go from here:

section 3

THE HONEST PART · SECTION 3 OF 4

Where to go from here.

keep going

Remember those podcast recommendations from chapter 06? Go listen to one and keep learning. Or better yet, put what you've built to work:

PASTE THIS INTO CLAUDE

I want to build a simple personal-assistant website I can run on my own computer with Ted Lasso as the host. Help me brainstorm what would actually be useful, then help me build it and understand what you need along the way.

You taught him to talk in chapter 04, and the brainstorming skill from chapter 05 will kick in. Time to put him to work.

CH.04 CALLBACK

You wrote a CLAUDE.md for Ted Lasso's voice. That context is still there, waiting.

CH.05 CALLBACK

The brainstorming skill kicks in automatically when the prompt calls for it.

THE POINT

Each chapter built on the last. Now you can build on all of them at once.

THE HONEST PART · SECTION 4 OF 4

One last thing.

The part this book can't teach you is the part that ends up mattering most: knowing when NOT to use it.

- *When to write the email yourself, because the relationship is the point.*
- *When to do the work by hand, because the thinking is the point.*
- *When to close the terminal, because you're tired and the next move can wait.*

The tool is the tool. You're the one deciding what's worth your time. That's the actual skill, and you already had it before you opened this book.

That's the book.

Go play.



WHERE YOU ARE · SECTION 4 OF 4

Now you can.

- 1 Install Claude Code on any computer that lets you install things
- 2 Make Claude do real work for you: write files, organize folders, draft the thing you've been procrastinating on
- 3 Teach Claude about a project with CLAUDE.md, so you stop re-explaining yourself every session
- 4 Install other people's skills with a couple of commands
- 5 Plug Claude into outside apps with MCP, when the built-in tools aren't enough

Seven chapters down. Two more when you need them.

UP NEXT • CHAPTER 8

08

Pull the plug.

Getting Claude Code off your computer should be as easy as putting it on. This chapter walks you through the full uninstall.



STARTS NEXT



YOU WILL DO

Uninstall the claude command and clean up your config folder.

ALL THREE OSES

Mac, Windows, and Linux each get their own step-by-step section.

NO HARD FEELINGS

Chapter 02 is still there if you change your mind.

*see you on the next page*honestly, **brandon**. still not an algorithm

repo • ch.08

CHAPTER 8 • CLEAN EXIT

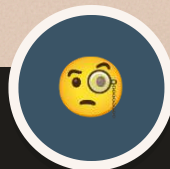
08

← no hard feelings.

Pull the plug.

You'll do

- ✓ Uninstall the claude command
- ✓ Remove your `~/.claude/` folder
- ✓ Clean up demo folders (optional)
- ✓ Cancel your Anthropic subscription



Honest questions

- 1 Does uninstalling delete my account?
- 2 What happens to my `~/.claude/` data?
- 3 Can I reinstall later?

CONCEPT · SECTION 1 OF 7

First, a word about what got installed.

(take what you need, skip the rest.)

You tried it. Maybe it's not for you. Maybe you're moving to a new machine and want a clean install. Maybe you're switching tools. Whatever the reason, getting Claude Code off your computer should be as easy as putting it on.

No hard feelings. This chapter walks you through the full uninstall: the CLI, the configs, the demo folders, the optional supporting stuff. Take what you need, skip what you don't.

When you went through chapter 02 and the chapters after it, a few things landed on your machine. Removing them one at a time, in order, leaves your machine back at neutral.

- **Claude Code itself:** the `cClaude` command, installed via npm.
- **Your `~/ .claude/` folder:** holds your memory, installed plugins, MCP registrations, skills, and Claude Code settings.
- **Demo folders:** `~/claude-experiments/packing-list/`, `writing/`, `brainstorm/` from chapters 03-06.
- **Node.js:** the runtime Claude Code is built on. Only remove this if you didn't have it before chapter 02.
- **An Anthropic subscription:** Pro, Max, or similar. A billing relationship with Anthropic that you cancel separately.



THE ORDER

CLI first, then config folder, then optional cleanup. Do it in this order and you won't miss anything.



YOUR ACCOUNT

Uninstalling the CLI does NOT delete your Anthropic account. That's a separate step covered in section 5.



REINSTALL LATER?

Yes. Chapter 02 is right where you left it.

SETUP · SECTION 2 OF 7

Mac.

open Terminal

STEP 1 · UNINSTALL THE CLI

```
bash
$ npm uninstall -g @anthropic-ai/claude-code
```

IF STEP 1 FAILS ("NOT INSTALLED", "COMMAND NOT FOUND")

Skip npm and delete the binary directly:

```
bash
$ which claude
$ rm $(which claude)
```

STEP 2 · REMOVE THE CONFIG FOLDER

```
bash
$ rm -rf ~/.claude
```

Wipes memory, installed plugins, MCP registrations, skill files, and settings. No trace left.

OPTIONAL: DEMO FOLDERS

```
bash
$ rm -rf ~/claude-
  experiments
```

OPTIONAL: REMOVE NVM +
NODE

```
bash
$ nvm uninstall --lts
$ rm -rf ~/.nvm
```

Then open `~/.zshrc` and delete the lines that mention nvm.

SETUP · SECTION 3 OF 7

Windows.

open Windows Terminal

STEP 1 · UNINSTALL THE CLI

Windows PowerShell

```
PS> npm uninstall -g @anthropic-ai/claude-code
```

If you see an error about running scripts being disabled, use `npm.cmd` instead:

Windows PowerShell

```
PS> npm.cmd uninstall -g @anthropic-ai/claude-code
```

IF STEP 1 STILL FAILS

Find and delete the binary directly:

Windows PowerShell

```
PS> where claude
```

```
PS> Remove-Item (Get-Command claude).Source
```

STEP 2 · REMOVE THE CONFIG FOLDER

Windows PowerShell

```
PS> Remove-Item -Recurse -Force $env:USERPROFILE\.claude
```

OPTIONAL: DEMO FOLDERS

Windows PowerShell

```
PS> Remove-Item -  
Recurse -Force  
$env:USERPROFILE\c  
laude-experiments
```

OPTIONAL: REMOVE NODE

Settings → Apps → find **Node.js** →
Uninstall. Skip if you use Node
elsewhere.

SETUP · SECTION 4 OF 7

Linux / WSL.

open your terminal

If you're on WSL, follow this section, not the Windows one.

STEP 1 · UNINSTALL THE CLI

```
bash
$ npm uninstall -g @anthropic-ai/claude-code
```

IF NPM FAILS ("NPM: COMMAND NOT FOUND", "NOT INSTALLED")

Your terminal might suggest `sudo apt install npm`. Ignore that. Delete the binary directly instead:

```
bash
$ rm $(which claude)
```

STEP 2 · REMOVE THE CONFIG FOLDER

```
bash
$ rm -rf ~/.claude
```

OPTIONAL: DEMO FOLDERS

```
bash
$ rm -rf ~/claude-
  experiments
```

OPTIONAL: REMOVE NODE

Depends on your distro. `apt remove nodejs`, `dnf remove nodejs`, or `nvm uninstall --lts`. Skip if you use Node elsewhere.

SETUP · SECTION 5 OF 7

One last thing.

all OSeS

Cancel your Anthropic subscription. This part doesn't happen in the terminal.

THREE STEPS

- 1 **Open:** `https://claude.ai` in any browser.
- 2 **Go to:** your avatar (top right) → **Settings** → **Plans**.
- 3 **Cancel:** or downgrade to free. You won't be charged from the next billing cycle.

Free claude.ai web chat still works. You just lose Claude Code access when the current paid period ends.

This does NOT delete your Anthropic account. If you want to delete the account itself: Settings → Account → delete.

*Done. Your machine is back to its pre-Claude state.
No judgment. If you change your mind later, chapter
02 is right where you left it.*

CONCEPT · SECTION 6 OF 7

What just happened.

(what you actually removed.)

You removed, in this order:

- 1 The Claude Code CLI:** The `cClaude` command no longer exists on your machine.
- 2 Your `~/.cClaude/` folder:** Memory, plugins, MCPs, skills, settings. Gone.
- 3 Demo folders (optional):** If you cleaned those up too, your filesystem is back where it was before chapter 02.
- 4 Your Anthropic subscription (optional):** Billing relationship ended. The free claude.ai web chat still works.

One thing this does NOT do: delete your Anthropic account. If you also want to delete the account, not just the subscription, go to claude.ai → Settings → Account → delete.

WHAT STAYS

Any files Claude wrote to your other folders. Those aren't tracked by Claude Code and weren't removed. They're just regular files.

REINSTALL

Chapter 02 is right where you left it. The whole process takes about ten minutes.

GROUND RULES · SECTION 7 OF 7

In case of emergency.

the damn thing won't go away?

Probably one of these.

SYMPTOM	WHAT IT MEANS	FIX
<code>claude --version</code> still works after uninstall	Leftover binary somewhere.	Run which claude (Mac/Linux) or where claude (Windows) to find it. Delete the file.
<code>npm uninstall -g</code> says "not installed"	npm doesn't know about it. Different install method.	Use the which claude + <code>rm \$(which claude)</code> fallback from step 1.
Removed <code>~/.claude/</code> but Claude Code is still configured	A project folder has its own <code>.claude/</code> directory.	Delete <code>.claude/</code> from inside that project folder too.
Anthropic still charged you next month	Cancellation didn't go through on their end.	Email Anthropic support. Check Settings → Plans shows "Free."

WHERE YOU ARE · SECTION 7 OF 7

Now you can.

1 Start completely fresh with chapter 02 if you change your mind

2 Move on with no clutter left on your machine

3 Tell people "yeah, you can uninstall it in like 5 commands" with first-hand credibility

Eight down. One more: the questions you came here with.

UP NEXT • CHAPTER 9

09

*The questions
you came here with.*

Is my data safe? Will it replace me? Am I getting dependent? The honest answers, all in one place.



STARTS NEXT



YOU WILL GET

Honest answers to the five questions most people are afraid to ask out loud.

NO LECTURES

No "embrace transformation." Just what's actually true and what it means for you.

DIFFICULTY

No new commands. Just the honest conversation the book has been building toward.

*see you on the next page*honestly, **brandon**. still not an algorithm

repo • ch.09

CHAPTER 9 • THE HONEST PART

09

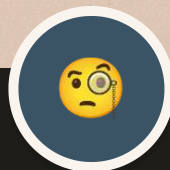
← the real questions.

*The questions
you came here with.*



You'll learn

- ✓ Is my data safe?
- ✓ When it screws up
- ✓ Will it replace me?
- ✓ Getting dependent
- ✓ Burning out



Honest questions

- 1 What actually goes to Anthropic's servers
- 2 How hallucinations work and how to catch them
- 3 An honest look at job risk, dependency, and burnout

— THE HONEST PART

Is my data safe?

The honest answer is "yes, mostly, and here's where you should still pay attention." Here's what actually happens.

STAYS ON YOUR MACHINE

Your project files, `~/ .claude/` folder, session transcript, code Claude writes to disk. Claude can't reach the internet without your approval.

LEAVES YOUR MACHINE

Your prompt, the contents of any files Claude needed to read, and Claude's output. It has to. The model lives on Anthropic's servers, not yours.

TRAINING ON YOUR CHATS?

No, not by default for Pro, Max, and API users. Free users are asked to opt in or out. Policies change. Check anthropic.com/legal/privacy before you assume.

THE ACTUAL RISK

What you paste in. Don't paste API keys, passwords, customer data, anything covered by NDA or HIPAA, or anything you'd regret having in a log.

TL;DR

Claude Code sends your prompts and relevant file contents to Anthropic. Paid plans don't train on them. The real risk is what you paste.

FULL BREAKDOWN

anthropic.com/legal/privacy



— THE HONEST PART

When it screws up?

Claude is going to confidently tell you something false. Sometimes spectacularly false. With perfect grammar and zero hesitation. This is the most important thing to know before you trust it with anything that matters.

What hallucinations look like: citations that don't exist, made-up commands, quotes attributed to people who never said them, confident wrong math, plausible-sounding biographical details that are wrong, filenames or paths that don't exist in your project.

HIGH RISK

Legal or financial stakes, medical info (dosages especially), citations you'll publish, commands Claude wants to run, specific dates and numbers, anything you'll send to another human as fact.

LOWER RISK

Brainstorming, reformatting content, explaining concepts you already understand, first drafts you'll edit anyway.

How to catch it: ask "how do you know that?" Verify the one or two claims your decision actually rests on. Read what it wrote, line by line, before you click yes.

TL;DR

Claude will confidently make things up. Read everything it wants to change. Verify anything that matters. The check is you.

— THE HONEST PART

Will it replace me?

The honest answer doesn't fit in a sentence. The doomers say the end times are here. The consultants say you'll be fine if you "embrace transformation." Neither knows your specific situation.

Direct-replacement risk: Some jobs are already being affected. Right now. Tier-1 customer service, copy editing, junior coding work, transcription, first-draft copywriting, basic graphic design, data entry, paralegal first-pass document review. If you're in one of these, the honest move is to climb up the value chain or pivot toward judgment, relationships, or physical presence.

Second-order risk (even if your job is "safe"): When some jobs disappear, those workers apply for the remaining ones. Hiring tightens everywhere. AI-augmented output becomes the new baseline. Even "safe" roles see pay compression as the broader job market loosens.

You're not waiting for someone to translate "AI" for you. That's already most of the game.

TL;DR

Some roles are already affected. The people who do well are the ones who understand the tool well enough to use it and catch it.

— THE HONEST PART

Am I getting dependent?

You reach for Claude before you've thought about the problem. You feel stuck without it. You can't remember how you used to start a draft. Congratulations: that's the slide.

What dependency looks like: reaching for Claude before you've thought about the problem, panic when it's down, forgetting how you used to do things you used to do, outsourcing things you actually wanted to do yourself.

The two kinds: healthy reliance (you use it because it's the right tool; you could do without; you just don't want to) versus unhealthy dependency (you can't do without; the skill has gone soft; the choice isn't yours anymore).

Skills that go soft first: writing your own first sentence, holding a thought long enough to develop it, sitting with the discomfort of not knowing, knowing when something is good without asking, trust in your own judgment.

*Pick three things that stay yours. Notice the reach. **The choice stays yours.***

TL;DR

The slide is slow. Pick three things to keep manual, on purpose. The goal isn't using it less. It's staying capable of doing without it.

— THE HONEST PART

What about burning out?

AI is everywhere now, whether you use it or not. Even if you never open Claude, the people around you are. The result is a kind of tiredness most of us don't have a name for yet.

VERIFICATION FATIGUE

Constant fact-checking exhausts your brain in a way doing the work yourself doesn't.

DECISION FATIGUE

"Take it, edit it, redo it, or scrap it?"
Multiplied across a full day.

HOLLOW-CONTENT FATIGUE

Scrolling past posts that all sound like the same person wrote them. Because they kind of did.

IDENTITY DRAG

Finishing a piece of writing that doesn't feel like you wrote it.

The sameness problem: same tools, same prompts, same outputs. Emails sound the same. Decks look the same. Articles read the same. People who get pulled in stop standing out and stop standing for anything.

Stay weird on purpose. The model is a tool, not a co-author.

TL;DR

AI burnout is real. The antidote is weirdness, on purpose. The automated world rewards both speed and distinction. Only you can supply the second one.

THE QUESTIONS, ANSWERED

Is my data safe?

Mostly yes. Paid plans don't train on your conversations.
The real risk is what you paste in.

When it screws up?

Verify anything that matters. Read what it writes before
you approve it. The check is you.

Will it replace me?

Some roles are already affected. The people who do well
understand the tool well enough to use it and catch it.

Getting dependent?

Pick three things to keep manual. Stay capable of doing
without it. The choice stays yours.

Burning out?

Stay weird on purpose. The tool handles speed. Only you can
supply distinction.

That's the book. Go play.

honestly, **brandon**. still not an algorithm.



REFERENCE • CHEAT SHEET

10

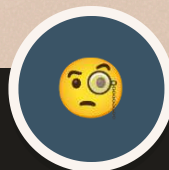
← tape it to your wall.

The stuff you
actually need.



You'll look up

- ✓ Open and close Claude
- ✓ Navigate your terminal
- ✓ Slash commands
- ✓ Plugins and MCPs
- ✓ What to do when broken



Honest questions

- 1 What's the one slash command you actually need?
- 2 What do you do when Claude stops responding?
- 3 Do you have to memorize any of this?

In the terminal.

type these outside Claude

OPEN + CLOSE

START CLAUDE

```
claude
```

CHECK INSTALL

```
claude --version
```

EXIT CLEANLY

```
/exit
```

FORCE QUIT

```
Ctrl+C      twice
```

NAVIGATE YOUR COMPUTER

WHERE AM I?

```
pwd
```

WHAT'S IN HERE?

```
ls
```

STEP INTO A FOLDER

```
cd folder-name
```

GO UP ONE LEVEL

```
cd ..
```

GO HOME

```
cd ~
```

MAKE A FOLDER

```
mkdir folder-name
```

Native on Mac, Linux, and WSL. PowerShell supports all of these as aliases — they work, output looks slightly different.

REFERENCE · SECTION 2 OF 4

Inside Claude.

type these at the prompt

SLASH COMMANDS

SEE WHAT'S AVAILABLE

`/help`

CHANGE THE THEME

`/theme`

SEE PERMISSIONS

`/permissions`

SEE CONNECTED APPS

`/mcp`

RELOAD PLUGINS

`/reload-plugins`

EXIT

`/exit`

PLUGINS — BUNDLES OF SKILLS

ADD A MARKETPLACE

`/plugin marketplace add
<owner>/repo>`

INSTALL A PLUGIN

`/plugin install
<name>@<marketplace>`

UNINSTALL A PLUGIN

`/plugin uninstall
<name>@<marketplace>`

REMOVE A MARKETPLACE

`/plugin marketplace remove
<name>`

Always run `/reload-plugins` after installing or removing.

Connect and fix.

don't throw the laptop

MCPs — CONNECT CLAUDE TO OTHER APPS

ADD AN MCP

```
claude mcp add  
--transport http  
<name> <url>
```

LIST YOUR MCPs

```
claude mcp list
```

REMOVE AN MCP

```
claude mcp  
remove  
<name>
```

Then run `/mcp` inside Claude to confirm it's connected.

WHEN SOMETHING'S BROKEN

claude: command not found



Close and reopen your terminal

A slash command does nothing



Run `/reload-plugins`

Permission error on install



Don't use `sudo` — see chapter 02

Claude wrote in the wrong folder



`/exit`, `cd` to the right folder, claude again

MCP shows "failed"



Remove it, add it again

Anything else



Copy the error. Ask Claude what it means.

Two things to never forget.

1. **Read what Claude wrote before you click yes.**

Even when half of it might as well be in Greek. Look anyway.

2. **You can always `/exit` and start over.**

Nothing is permanent.

you actually did it.

honestly, **brandon**. still not an algorithm.

